

Physical AI 보고서



Physical AI
로봇학부 2023741071
이선경

1. 프로젝트 목표

기존 Raccoonbot OpenVLA 환경의 한계(원통형 실린더 , **grasp** 작업)만 수행 가능했던 구성에서, 더 안정적이고 범용성 있는 명령을 이해하고 처리할 수 있는 프로젝트 코드로 발전시키는 것이 이번 프로젝트의 목표이다.

이를 위해 데이터셋 확장과 코드 개선 두가지 측면에서 프로젝트를 수행하며 레퍼런스 코드에 추가 데이터셋 생성부터 학습 및 파인튜닝, 추론 과정을 거쳐 모델과 데이터셋을 발전시키는 과정에 참여해보고, 나아가 시뮬레이션이 아닌 실제 로봇환경에서도 시뮬레이션과 동기화되어 작업을 수행할 수 있도록 한다.

2. Multi-task Dataset Expansion

2.1 new objects : cube(box)

Mujoco 시뮬레이션 환경상에 존재하던 기존 **red/ blue** 실린더에 갈색 큐브 객체를 추가하였다. 총 5개의 오브제는 데이터 생성과정에서 함께 생성되며 모두 랜덤한 위치에 배치될 수 있도록 수정하였다. 질량이나 **friction**은 원기둥보다는 조금 더 크지만 거의 비슷하게 설정하였다.

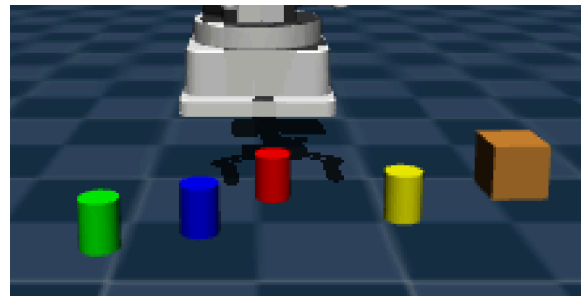
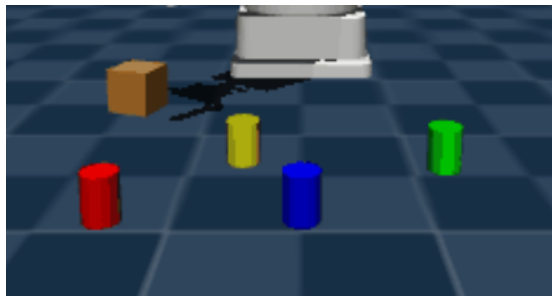


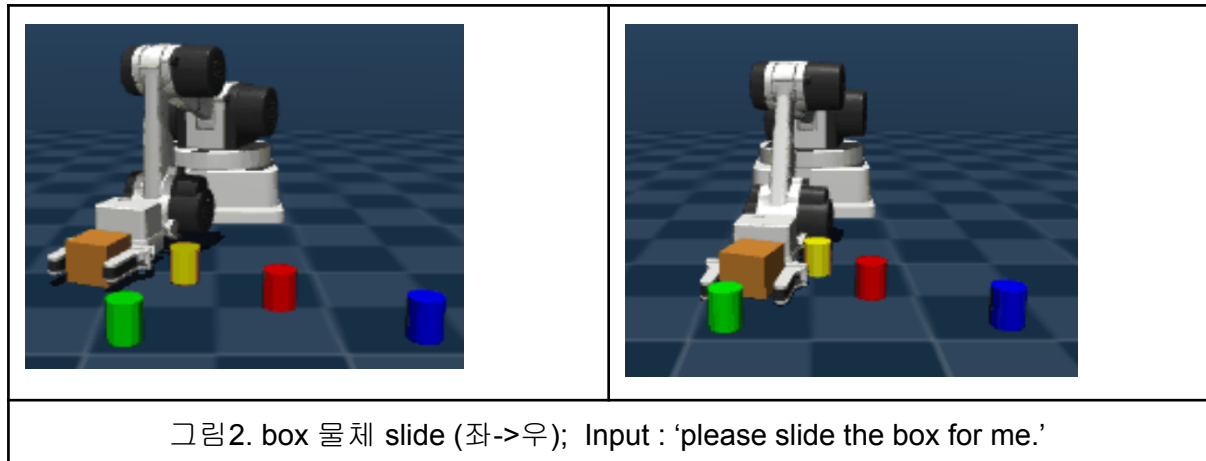
그림 1. 랜덤하게 배치된 Mujoco 내부의 objects

2.2 new tasks : slide(box)

기존 **grasp**만 가능하던 레퍼런스 코드에 **push task**를 추가하였다.

학습 초반 문제 상황 : 데이터 수집과 학습 초반에는 **grasp**와 **push**를 별개의 작업으로 생각하고, 단지 물체를 ‘미는’행동에 초점을 맞추어 **타겟 물체의 거리 이동에 초점**을 맞추었다. 이 경우 로봇 엔드이펙터 부근의 4축모터(손목)이 위쪽으로 들리면서 물체를 넘어트리고 원하던 대로 미는 모션보다 **굴러거나 넘어지거나, 그리퍼가 이미 닫힌 상태에서 물체를 치는 형태**의 모션이 관측 되었다.

문제 해결 : 타겟 물체의 **grasp** 작업을 먼저 수행한 이후 **z축**높이를 고정시킨채 이동하도록 수정을 진행하였다. 수정 결과, 그리퍼 안쪽으로 **물체를 잡고 이동**할 수 있도록 해당 객체의 **grasp**를 수행한 뒤 중앙에 잡혀 있는 상태에서 위아래로 손목이 꺾이지 않고, 물체가 굴러가지 않는 채로 **미끄러지듯 움직일** 수 있게졌다.



2.3 Diverse Language Instructions:

기존의 "grasp the {color} cylinder" 와 같이 color 속성만 가변적인 코드에서, task를 추가하며 grasp, push 등 동사와 객체를 추가하며 cylinder, box와 같은 noun 속성을 추가할 수 있었다. 그러나 이 뿐 만이 아닌, 실제 사용자와 대화상황이라면 간단한 문장구조보다 더 복잡한 수식어가 붙어도 해석이 가능해야 한다고 생각했다.

따라서 문장 구성에 맞는 기본 뼈대 템플릿을 여러 종류 만들고,

{verb},{noun},{prefix},{color}등의 항목을 만들어 새로운 문장을 생성 가능하도록했다.

기존 4가지만 가능했던 문장에서 20개 내지의 단어들로 총 2475개의 문장을 생성해 낼 수 있어졌다.

grasp + box: verbs 5 × prefixes 5 × nouns 5 × templates 3 = 375

grasp + cylinder: verbs 5 × prefixes 5 × nouns 4 × templates 3 × color 4 = 1200

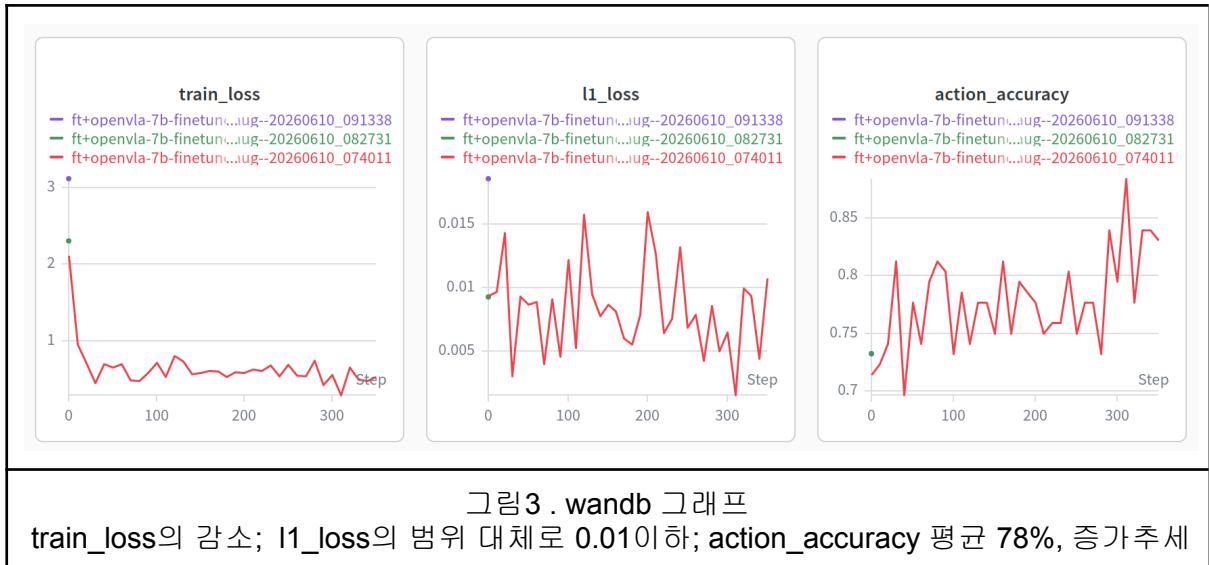
push + box: verbs 3 × prefixes 4 × nouns 5 × goals 5 × templates 3 = 900

예시 문장 조합:

Please grab the cube. [template1]
Carefully grasp the blue cylinder. [template1]
Can you catch the red item for me. [template2]
Slide the block toward the goal. [template3]
Gently push the box to the target spot. [template4]

2.4 데이터셋 생성 및 학습:

데이터셋 추가 생성과정에서 box와 새로운 언어 명령으로 동작하는 데이터를 5개의 아이템과 push, grasp 조합의 각각 150개의 에피소드씩 모아 추가 데이터셋을 생성하였다. 모아진 데이터와 기존 데이터셋을 함께 사용하여 OpenVLA 모델을 LoRA 방식으로 파인튜닝하였다. 초반 학습 시에는 wandb를 사용해 train loss가 감소하는것을 확인한 뒤 학습을 진행하였다. (wandb 용량 부족 문제로 전체 학습을 관리해 볼수 없어 학습 초반에 사용하였다.)



학습에는 openvla/openvla-7b 모델을 기반으로 사용하였고, LoRA rank는 32, batch size는 8, gradient accumulation step은 2로 설정하였다. 따라서 실제 effective batch size는 16이다.

학습 명령어에서는 최대 학습 스텝을 30000 step으로 설정하였으며, adapter checkpoint는 10000 step마다 저장되도록 설정하였다. 이후 추론 단계에서는 저장된 adapter 중 raccoon_pick_place_lr0.0005_0611_0925_raccoon-eef-v100--20000_adapter를 불러와 사용하였다. 즉, 최종 추론에 사용한 모델은 20000 step까지 학습된 adapter이다.

```
#####
# terminal command
#####
cd /data/lisk/Raccoonbot_Openvla/openvla
CUDA_VISIBLE_DEVICES=0 python openvla_server.py \
  --model_path
/data/lisk/Raccoonbot_Openvla/openvla/openvla-adapter-tmp/raccoon_pick_place_lr0.000
5_0611_0925_raccoon-eef-v100--20000_adapter \
  --default-unnorm-key raccoon_pick_place \
  --host 0.0.0.0 \
  --port 8000 \
  --device cuda

#####
# finetune2 는 개인 wandb 사용하도록 되어있음
#####
CUDA_VISIBLE_DEVICES=0 \
torchrun --standalone --nnodes 1 --nproc-per-node 1 vla-scripts/finetune2.py \
  --vla_path openvla/openvla-7b \
  --data_root_dir /data/lisk/Raccoonbot_Openvla/combined_dataset \
  --dataset_name raccoon_pick_place \
```

```
--run_root_dir /data/lisk/Raccoonbot_Openvla/openvla/openvla-runs \
--adapter_tmp_dir /data/lisk/Raccoonbot_Openvla/openvla/openvla-adapter-tmp \
--lora_rank 32 \
--batch_size 8 \
--grad_accumulation_steps 2 \
--learning_rate 5e-4 \
--max_steps 30000 \
--save_steps 10000 \
--save_adapter_checkpoints_only true \
--run_id_note raccoon-eef-v100
```

3. Code Improvement

3.1 Speed up :

이미지 전송 방식 변경 : 추론 시 이미지 전송 방식을 PNG로 base64 인코딩하던 방식에서 JPEG로 인코딩 하고 quality를 조절 가능하게 하였다.

추가 옵션 설정 기능 :

프레임 저장 끄기 옵션 '**--no_save_frames**' 추가, 매 step마다 무조건 이미지를 저장하지 않아 실제 로봇 사용에 유리함.

이미지 전송 방식 변경의 '**--image_quality**' 옵션으로 테스트 해본 결과 20은 로봇이 인식 성능이 낮아져 속도는 빠르지만 진동하고, 50의 경우 정확하지만 느린 편으로 안정적이되 가장 빠른 30으로 추론을 진행하였다.

적용 전 후 비교 결과, 동일 seed를 주고 red cylinder grasp 상황에서 테스트해본 결과, 한 에피소드를 실행하는 데 기존 3분 18초에서 2분 5초까지 감소하였다.

#####

테스트 당시 8000포트가 이미 점유되어있다는 에러로 양 측을 8001로 바꾸어 진행하였다.

#####

```
python openvla_multicolor_client_real_robot.py --server_url http://127.0.0.1:8001
--target_color green --seed 42 --image_quality 30 --no_save_frames
--use_real_robot --use_viewer
[CLEANUP] removed 0 existing image files from rollout_outputs/episode_000001
Raccoon[0] Connected: /dev/ttyACM0 D5:BF:DA:6B:6F:D6
[REAL_ROBOT] 하드웨어 연결 성공
[DEBUG] current ee from get_ee_pose = (0.00, 14.74, 9.44) cm
[DEBUG] IK(current ee) = [-0.005280205391976754, -10.001870609331936,
-140.00075796191655, 60.00262857124849]
[SCENE] instruction='grasp the green cylinder' | target_color='green' | target_xy=(0.057,
0.172) | objects={'red': {'body_name': 'target_object', 'xy': [-0.054552255643044625,
0.20991263083142514], 'yaw': -0.6851542519228805}, 'blue': {'body_name':
'target_object_blue', 'xy': [-0.025840395153483756, 0.24340884899637416], 'yaw':
```

```
0.2259828021766146}, 'green': {'body_name': 'target_object_green', 'xy':  
[0.05721286105539078, 0.17153022694079914], 'yaw': -0.077933586511017}, 'yellow':  
{'body_name': 'target_object_yellow', 'xy': [0.07171958398227649,  
0.22276312261534276], 'yaw': -0.6374647312682433}}  
[INFER] server request time: 0.411s | image_quality=30  
[REAL_ROBOT] target_cm=[0.5, 15.01, 9.36] | joint_deg=[-1.91, -11.81, -138.38, 60.19] |  
gripper=open  
[000] OK | final_delta=[0.005, 0.0028, -0.0006] | move=[0.005, 0.0028, -0.0006] |  
target=[0.005, 0.1501, 0.0936] | gripper=0.0 | retries=0  
[INFER] server request time: 0.328s | image_quality=30
```

3.2 Add logs or better visualization :

추론 성공률 분석 **.sh** 파일 : 추론을 돌릴 때, 한번에 원하는 횟수만큼 돌리고, 그때 어떤 input을 사용했고 그 명령이 들어갔을때 성공률은 어떠한지 분석해줄수 있는 **shell**파일을 작성했다.

```
=====
Batch Rollout & Advanced Condition Logging
Target Episodes: 1
Analysis Report: ./Isk/OpenVLA/check_my_episode.csv
=====
[12:03:11] Episode 000001/1 시작...
[✓ LIFT_SUCCESS] Color: red | Verb: grasp | Noun: cylinder | Goal: N/A
      Result: 성공 | 누적성공률: 100% (1/1)
=====
      배치 롤아웃 조건별 최종 통계
=====
총 에피소드 수 : 1회
총 성공 횟수   : 1회
최종 전체 성공률: 100%

TARGET COLOR 통계 리포트
조건 이름      | 총 호출수   | 성공률
-----
red            | 1           | 100%

COMMAND VERB 통계 리포트
조건 이름      | 총 호출수   | 성공률
-----
grasp          | 1           | 100%

OBJECT NOUN 통계 리포트
조건 이름      | 총 호출수   | 성공률
-----
cylinder       | 1           | 100%

PUSH GOAL 통계 리포트
조건 이름      | 총 호출수   | 성공률
-----
```

```
=====
결과 CSV 저장 완료: ./lsk/OpenVLA/check_my_episode.csv
=====
```

또한 images로 저장된 이미지 데이들을 gif, mp4 형태로 렌더링해 변환해주는 .py 코드를 작성해 데이터셋이나 추론 결과를 시각화하여 확인할 수 있었다.

```
#####
```

```
#해당 코드는 원하는에피소드 경로를 넣어 변환할 수 있게 되어있다.
```

```
#####
```

```
python3 make_video.py
```

```
#####
```

```
# 뒤에 인자를 추가하여 몇개의 에피소드, 시각화 여부, 시드 고정, 실제 로봇데이터 사용 등을 제어 할 수 있다.
```

```
#####
```

```
./batch_rollout.sh 50 --use_viewer --seed 42 --use_real_robot
```

3.3 Real Robot Operation :



그림4. 실제 Raccoonbot grasp task; “Can you grasp the red cylinder”

시뮬 - 실제 로봇 동기화 : **3.1**을 수행하며 동시에 시뮬레이션상에서 추론하고 있는 로봇을 그대로 동일한 동작을 수행할 수 있도록 Raccoonbot을 연결해 확인해보았다. 현재는 시뮬레이션 상의 이미지를 input으로 받아와 action을 생성하지만, 추후 외부 카메라를 설치할 수 있다면 현실세계의 블록을 인식해 현실의 로봇이 움직일 수 있도록 input 이미지와 좌표계를 수정할 수도 있을것이다.

4. Result

기존 레퍼런스 코드에서 이번 프로젝트를 진행하며 여러 개선점들과, 그 과정에서 추가적으로 해결해야 할 문제들을 만나며 VLA란 무엇이고, PhysicalAI는 무엇인지를 이론을 넘어 실습을 통해 더 깊이 이해할 수 있었다. 글로만 배울 때는 몰랐던 내용이 실제 로봇을 구동하고 현실세계의 task를 해결하는 문제로 예시를 드니 더 쉽게 이해할 수 있었다.

이번 프로젝트에서 개선한 부분은 아래와 같다.

1. grasp only -> push task 추가
2. cylinder only -> box 추가
3. language prompt 확장
4. 속도 향상
5. 로그, 비주얼라이제이션 개선
6. 시뮬레이션과 실제 로봇 동기화 테스트

추론을 할 때 위치를 잘 인식하지 못하고 새로운 상황에 빠르게 대응하지 못하는 모습을 보며, 단순 성공 여부 뿐만 아니라 로봇이 추론해 온 경로, 시행 시간, 데이터 분포, 환경적 변화 등 다양한 요인을 고려해야 함을 알았다. 추가 학습해 넣은 **task**의 시도 양에 비해 해당 **task**를 배운 **train**이 상대적으로 학습량이 절대적으로 적다보니 새로운 추론 상황에서 잘 적응하지 못하는 모습을 확인할 수 있었다.

또한 제한된 서버를 함께 나누어 학습하다보니 절대적인 학습 시간의 부족으로 적은 시간동안 최대한 많이 확인해보고자 언어 **input**의 경우의 수를 늘리는 것과 새로운 큐브 모델의 도입을 동시에 적용하였는데, 이를 분리하여 기존 데이터셋에 따로 따로 추가해 학습을 돌렸다면 기존과 해당 기능 사이의 성능 상관관계를 파악하고 실패 원인을 파악하기 수월했을것 같다.

본 프로젝트에서는 실제 **VLA** 파이프라인을 따라 데이터 수집 및 생성부터 변환, 학습, 추론 전 과정과 더불어 실제 로봇에도 적용해보며 **VLA**를 실제 로봇에 적용하는 물리적 세계에서 **PhysicalAI**의 양상을 확인할 수 있었던 점에서 의의가 있다. 또한 **RaccoonBot** 환경에서 자연어를 통한 작업 수행에 기반이 될 수 있을것이다. 특히 데이터의 품질 뿐만 아니라, 데이터들의 분포와 관리, 학습하는 비율과 방법도 실제 수행 성능에 큰 영향을 미치는 중요성을 체감할 수 있던 경험이었다.